

A Step-by-step Introduction to the Implementation of Automatic Differentiation

Yu-Hsueh Fang^{*1}, He-Zhe Lin^{*1}, Jie-Jyun Liu¹, and Chih-Jen Lin^{1,2}

¹National Taiwan University

{d11725001, r11922027, d11922012}@ntu.edu.tw

cjlin@csie.ntu.edu.tw

²Mohamed bin Zayed University of Artificial Intelligence

chihjen.lin@mbzuai.ac.ae

December 4, 2025

Abstract

Automatic differentiation is a key component in deep learning. This topic is well studied and excellent surveys such as Baydin et al. (2018) have been available to clearly describe the basic concepts. Further, sophisticated implementations of automatic differentiation are now an important part of popular deep learning frameworks. However, it is difficult, if not impossible, to directly teach students the implementation of existing systems due to the complexity. On the other hand, if the teaching stops at the basic concept, students fail to sense the realization of an implementation. For example, we often mention the computational graph in teaching automatic differentiation, but students wonder how to implement and use it. In this document, we partially fill the gap by giving a step by step introduction of implementing a simple automatic differentiation system. We streamline the mathematical concepts and the implementation. Further, we give the motivation behind each implementation detail, so the whole setting becomes very natural.

1 Introduction

In modern machine learning, derivatives are the cornerstone of numerous applications and studies. The calculation often relies on automatic differentiation, a classic method for efficiently and accurately calculating derivatives of numeric functions. For example, deep learning cannot succeed without automatic differentiation. Therefore, teaching students how automatic differentiation works is highly essential.

Automatic differentiation is a well-developed area with rich literature. Excellent surveys including Chinchalkar (1994), Bartholomew-Biggs et al. (2000), Baydin et al. (2018) and Margossian

^{*}These authors contributed equally to this work

(2019) review the algorithms for automatic differentiation and its wide applications. In particular, Baydin et al. (2018) is a comprehensive work focusing on automatic differentiation in machine learning. Therefore, there is no lack of materials introducing the concept of automatic differentiation.

On the other hand, as deep learning systems now solve large-scale problems, it is inevitable that the implementation of automatic differentiation becomes highly sophisticated. For example, in popular deep learning systems such as PyTorch (Paszke et al., 2017) and Tensorflow (Abadi et al., 2016), at least thousands of lines of code are needed. Because of this, many places of teaching automatic differentiation for deep learning stop at the basic concepts. Then students fail to sense the realization of an implementation. For example, we often mention the computational graph in teaching automatic differentiation, but students wonder how to implement and use it. In this document, we aim to partially fill the gap by giving a tutorial on the basic implementation.

In recent years, many works^{1,2,3,4,5} have attempted to discuss the basic implementation of automatic differentiation. However, they still leave room for improvement. For example, some are not self-contained – they quickly talk about implementations without connecting to basic concepts. Ours, which is very suitable for the beginners, has the following features:

- We streamline the mathematical concepts and the implementation. Further, we give the motivation behind each implementation detail, so the whole setting becomes very natural.
- We use the example from Baydin et al. (2018) for the consistency with past works. Ours is thus an extension of Baydin et al. (2018) into the implementation details.
- We build a complete tutorial including this document, slides and the source code at <https://www.csie.ntu.edu.tw/~cjlin/papers/autodiff/>.

2 Automatic Differentiation

There are two major modes of automatic differentiation. In this section, we introduce the basic concepts of both modes. Most materials in this section are from Baydin et al. (2018). We consider the same example function

$$y = f(x_1, x_2) = \log x_1 + x_1 x_2 - \sin x_2.$$

¹<https://towardsdatascience.com/build-your-own-automatic-differentiation-program-6ecd585eec2a>

²<https://sidsite.com/posts/autodiff>

³<https://mdrk.io/introduction-to-automatic-differentiation-part2/>

⁴https://github.com/dlsyscourse/lecture5/blob/main/5_automatic_differentiation_implementation.ipynb

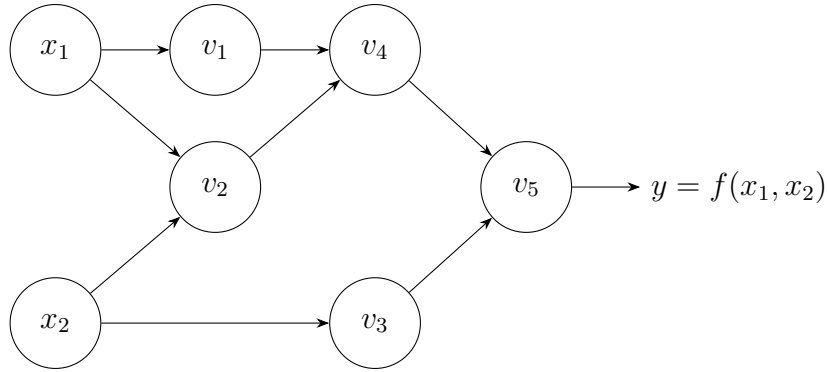
⁵<https://github.com/karpathy/micrograd>

2.1 Forward Mode

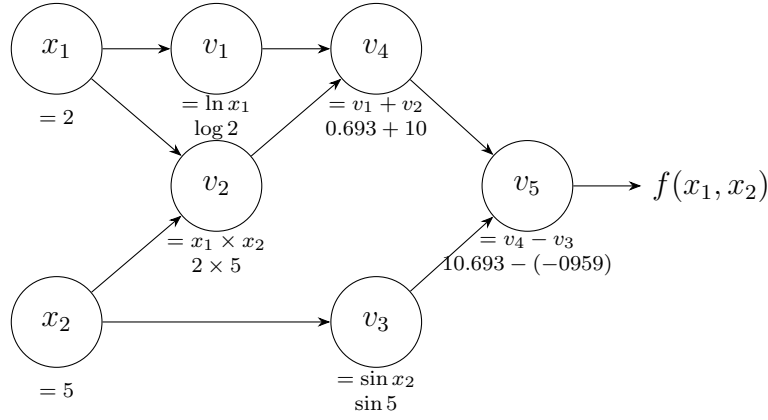
First, we discuss the forward mode. Before calculating the derivative, let us check how to calculate the function value. Assume that we want to calculate the function value at $(x_1, x_2) = (2, 5)$. Then, in the following table, we have a forward procedure.

x_1		$= 2$
x_2		$= 5$
v_1	$= \log x_1$	$= \log 2$
v_2	$= x_1 \times x_2$	$= 2 \times 5$
v_3	$= \sin x_2$	$= \sin 5$
v_4	$= v_1 + v_2$	$= 0.693 + 10$
v_5	$= v_4 - v_3$	$= 10.693 + 0.959$
y	$= v_5$	$= 11.652$

We use variables v_i to record the intermediate outcomes. First, we know $\log$ function is applied to x_1 . Therefore, we have $\log(x_1)$ as a new variable called v_1 . Similarly, there is a variable v_2 , which is $x_1 \times x_2$. Each v_i is related to a simple operation. The initial value of this member is zero when a node is created. In the end, our function value at $(2, 5)$ is $y = v_5$. As shown in the table, the function evaluation is decomposed into a sequence of simple operations. We have a corresponding computational graph as follows:



Because calculating both v_1 and v_2 needs x_1 , x_1 has two links to them in the graph. The following graph shows all the intermediate results in the computation.



The computational graph tells us the dependencies of variables. Thus, from the inputs x_1 and x_2 we can go through all nodes for getting the function value $y = v_5$ in the end.

Now, we have learned about the function evaluation. But remember, we would like to calculate the derivative. Assume that we target at the partial derivative $\partial y / \partial x_1$. Here, we denote

$$\dot{v} = \frac{\partial v}{\partial x_1}$$

as the derivative of the variable v with respect to x_1 . The idea is that by using the chain rule, we can obtain the following forward derivative calculation to eventually get $\dot{v}_5 = \partial f / \partial x_1$.

$\dot{x}_1$	$= \partial x_1 / \partial x_1$	$= 1$
$\dot{x}_2$	$= \partial x_2 / \partial x_1$	$= 0$
$\dot{v}_1$	$= \dot{x}_1 / x_1$	$= 1/2$
$\dot{v}_2$	$= \dot{x}_1 \times x_2 + \dot{x}_2 \times x_1$	$= 1 \times 5 + 0 \times 2$
$\dot{v}_3$	$= \dot{x}_2 \times \cos x_2$	$= 0 \times \cos 5$
$\dot{v}_4$	$= \dot{v}_1 + \dot{v}_2$	$= 0.5 + 5$
$\dot{v}_5$	$= \dot{v}_4 - \dot{v}_3$	$= 5.5 - 0$
$\dot{y}$	$= \dot{v}_5$	$= 5.5$

The table starts from $\dot{x}_1$ and $\dot{x}_2$, which are $\partial x_1 / \partial x_1 = 1$ and $\partial x_2 / \partial x_1 = 0$. Based on $\dot{x}_1$ and $\dot{x}_2$, we can calculate other values. For example, let us check the partial derivative $\partial v_1 / \partial x_1$. From

$$v_1 = \log x_1,$$

by the chain rule,

$$\dot{v}_1 = \frac{\partial v_1}{\partial x_1} \times \frac{\partial x_1}{\partial x_1} = \frac{1}{x_1} \times \frac{\partial x_1}{\partial x_1} = \frac{\dot{x}_1}{x_1}.$$

Therefore, we need $\dot{x}_1$ and x_1 for calculating $\dot{v}_1$ on the left-hand side. We already have the value of $\dot{x}_1$ from the previous step ($\partial x_1 / \partial x_1 = 1$). Also, the function evaluation gives the value of x_1 . Then, we can calculate $\dot{x}_1 / x_1 = 1/2$. Clearly, the chain rule plays an important role here. The calculation of other $\dot{v}_i$ is similar.

2.2 Reverse Mode

Next, we discuss the reverse mode. We denote

$$\bar{v} = \frac{\partial y}{\partial v}$$

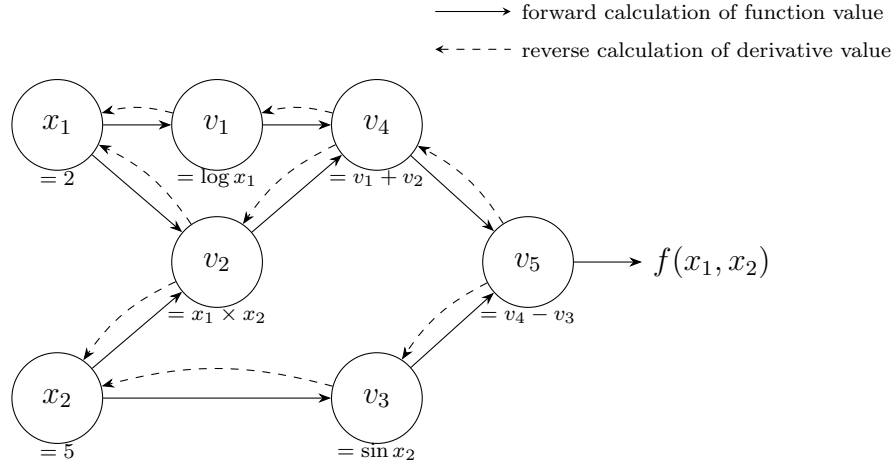
as the derivative of the function y with respect to the variable v . Note that earlier, in the forward mode, we considered

$$\dot{v} = \frac{\partial v}{\partial x_1},$$

so the focus is on the derivatives of all variables with respect to one input variable. In contrast, the reverse mode focuses on $\bar{v} = \partial y / \partial v$ for all v , the partial derivatives of one output with respect to all variables. Therefore, for our example, we can use $\bar{v}_i$'s and $\bar{x}_i$'s to get both $\partial y / \partial x_1$ and $\partial y / \partial x_2$ at once. Now we illustrate the calculation of

$$\frac{\partial y}{\partial x_2}.$$

By checking the variable x_2 in the computational graph, we see that variable x_2 affects y by affecting v_2 and v_3 .



This dependency, together with the fact that x_1 is fixed, means if we would like to calculate $\partial y / \partial x_2$, then it is equal to calculate

$$\frac{\partial y}{\partial x_2} = \frac{\partial y}{\partial v_2} \frac{\partial v_2}{\partial x_2} + \frac{\partial y}{\partial v_3} \frac{\partial v_3}{\partial x_2}. \quad (1)$$

We can rewrite Equation 1 as follows with our notation.

$$\bar{x}_2 = \bar{v}_2 \frac{\partial v_2}{\partial x_2} + \bar{v}_3 \frac{\partial v_3}{\partial x_2}. \quad (2)$$

If $\bar{v}_2$ and $\bar{v}_3$ are available beforehand, all we need is to calculate $\partial v_2/\partial x_2$ and $\partial v_3/\partial x_2$. From the operation between x_2 and v_3 , we know that $\partial v_3/\partial x_2 = \cos(x_2)$. Similarly, we have $\partial v_2/\partial x_2 = x_1$. Then, the evaluation of $\bar{x}_2$ is done in two steps:

$$\begin{aligned}\bar{x}_2 &\leftarrow \bar{v}_3 \frac{\partial v_3}{\partial x_2} \\ \bar{x}_2 &\leftarrow \bar{x}_2 + \bar{v}_2 \frac{\partial v_2}{\partial x_2}.\end{aligned}$$

These steps are part of the sequence of a reverse traversal, shown in the following table.

$\bar{x}_1$				= 5.5
$\bar{x}_2$				= 1.716
$\bar{x}_1$	= $\bar{x}_1 + \bar{v}_1 \frac{\partial v_1}{\partial x_1}$	= $\bar{x}_1 + \bar{v}_1/x_1$		= 5.5
$\bar{x}_2$	= $\bar{x}_2 + \bar{v}_2 \frac{\partial v_2}{\partial x_2}$	= $\bar{x}_2 + \bar{v}_2 \times x_1$		= 1.716
$\bar{x}_1$	= $\bar{v}_2 \frac{\partial v_2}{\partial x_1}$	= $\bar{v}_2 \times x_2$		= 5
$\bar{x}_2$	= $\bar{v}_3 \frac{\partial v_3}{\partial x_2}$	= $\bar{v}_3 \times \cos x_2$		= -0.284
$\bar{v}_2$	= $\bar{v}_4 \frac{\partial v_4}{\partial v_2}$	= $\bar{v}_4 \times 1$		= 1
$\bar{v}_1$	= $\bar{v}_4 \frac{\partial v_4}{\partial v_1}$	= $\bar{v}_4 \times 1$		= 1
$\bar{v}_3$	= $\bar{v}_5 \frac{\partial v_5}{\partial v_3}$	= $\bar{v}_5 \times (-1)$		= -1
$\bar{v}_4$	= $\bar{v}_5 \frac{\partial v_5}{\partial v_4}$	= $\bar{v}_5 \times 1$		= 1
$\bar{v}_5$	= $\bar{y}$	= 1		

To get the desired $\bar{x}_1$ and $\bar{x}_2$ (i.e., $\partial y/\partial x_1$ and $\partial y/\partial x_2$), we begin with

$$\bar{v}_5 = \frac{\partial y}{\partial v_5} = \frac{\partial y}{\partial y} = 1.$$

From the computational graph, we then get $\bar{v}_4$ and $\bar{v}_3$. Because v_4 affects y only through v_5 , we have

$$\bar{v}_4 = \frac{\partial y}{\partial v_4} = \frac{\partial y}{\partial v_5} \frac{\partial v_5}{\partial v_4} = \bar{v}_5 \frac{\partial v_5}{\partial v_4} = \bar{v}_5 \times 1$$

The above equation is based on that we already know $\partial y/\partial v_5 = \bar{v}_5$. Also, the operation from v_4 to v_5 is an addition, so $\partial v_5/\partial v_4$ is a constant 1. By such a sequence, in the end, we obtain

$$\frac{\partial y}{\partial x_1} = \bar{x}_1 \text{ and } \frac{\partial y}{\partial x_2} = \bar{x}_2$$

at the same time.

2.2.1 Extension to Vector-Valued Functions

The previous sections discussed functions with scalar variables. This is sufficient for gathering a basic understanding of automatic differentiation, but not enough to understand production-level implementations used in deep learning. In deep learning, we deal with functions of many variables. Consider a general function mapping $\mathbf{x} \in \mathbb{R}^n$ to $\mathbf{y} \in \mathbb{R}^m$:

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m.$$

For such a function, the derivative is not a single number, but a matrix known as the **Jacobian matrix**. The Jacobian contains the partial derivatives of all m outputs with respect to all n inputs:

$$J = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix} \quad (3)$$

To calculate this full matrix using the forward mode, we would need n passes (one for each input variable). Alternatively, calculating it row-by-row would require m passes. Therefore, the reverse mode is significantly more efficient than the forward mode when the number of outputs is much smaller than the number of inputs ($m \ll n$). In deep learning, we typically have a single scalar loss function ($m = 1$) and millions of parameters (n is large), making reverse mode the ideal choice.

2.2.2 Transposed Jacobian-vector Product

In optimization, we rarely need the full Jacobian matrix in isolation. Instead, a commonly used operation is the **transposed Jacobian-vector product**. Let r be a vector of size m (often representing the gradient of the loss with respect to the output y). We wish to calculate:

$$J^T r = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_1}{\partial x_n} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix} \begin{bmatrix} r_1 \\ \vdots \\ r_m \end{bmatrix} \quad (4)$$

By initializing the reverse mode calculation with $\bar{y} = r$, the reverse pass effectively computes the product $J^T r$ in a single pass. This operation propagates the gradient r backwards through the Jacobian to obtain the gradient with respect to the inputs. In the context of implementation (as seen in Section 6), this operation is referred to as the **Vector-Jacobian Product (VJP)**.

3 Implementation of Function Evaluation and the Computational Graph

With the basic concepts ready in Section 2, we move to the implementation of the automatic differentiation. For simplicity, we consider the forward mode. The reverse mode can be designed in a similar way. Consider a function $f : R^n \rightarrow R$ with

$$y = f(\mathbf{x}) = f(x_1, x_2, \dots, x_n).$$

For any given $\mathbf{x}$, we show the computation of

$$\frac{\partial y}{\partial x_1}$$

as an example.

3.1 The Need to Calculate Function Values

We are calculating the derivative, so at the first glance, function values are not needed. However, we show that function evaluation is necessary due to the function structure and the use of the chain rule. To explain this, we begin with knowing that the function of a neural network is usually a nested composite function

$$f(\mathbf{x}) = h_k(h_{k-1}(\dots h_1(\mathbf{x})))$$

due to the layered structure. For easy discussion, let us assume that $f(\mathbf{x})$ is the following general composite function

$$f(\mathbf{x}) = g(h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_k(\mathbf{x})).$$

We can see that the example function considered earlier

$$f(x_1, x_2) = \log x_1 + x_1 x_2 - \sin x_2 \tag{5}$$

can be written in the following composite function

$$g(h_1(x_1, x_2), h_2(x_1, x_2))$$

with

$$g(h_1, h_2) = h_1 - h_2,$$

$$h_1(x_1, x_2) = \log x_1 + x_1 x_2,$$

$$h_2(x_1, x_2) = \sin(x_2).$$

To calculate the derivative at $\mathbf{x} = \mathbf{x}_0$ using the chain rule, we have

$$\left. \frac{\partial f}{\partial x_1} \right|_{\mathbf{x}=\mathbf{x}_0} = \sum_{i=1}^k \left(\left. \frac{\partial g}{\partial h_i} \right|_{\mathbf{h}=\mathbf{h}(\mathbf{x}_0)} \times \left. \frac{\partial h_i}{\partial x_1} \right|_{\mathbf{x}=\mathbf{x}_0} \right),$$

where the notation

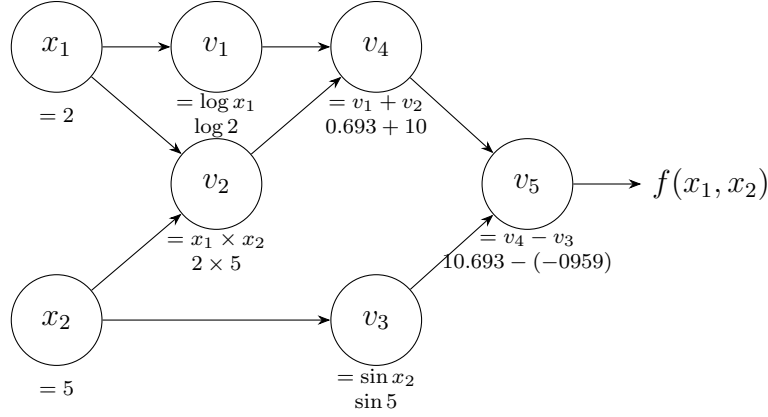
$$\left. \frac{\partial g}{\partial h_i} \right|_{\mathbf{h}=\mathbf{h}(\mathbf{x}_0)}$$

means the derivative of g with respect to h_i evaluated at $\mathbf{h}(\mathbf{x}_0) = [h_1(\mathbf{x}_0) \ \dots \ h_k(\mathbf{x}_0)]^T$. Clearly, we must calculate the inner function values $h_1(\mathbf{x}_0), \dots, h_k(\mathbf{x}_0)$ first. The process of computing all $h_i(\mathbf{x}_0)$ is part of (or almost the same as) the process of computing $f(\mathbf{x}_0)$. This explanation tells why for calculating the partial derivatives, we need the function value first.

Next, we discuss the implementation of getting the function value. For the function (5), recall we have a table recording the order to get $f(x_1, x_2)$:

x_1		$= 2$
x_2		$= 5$
v_1	$= \log x_1$	$= \log 2$
v_2	$= x_1 \times x_2$	$= 2 \times 5$
v_3	$= \sin x_2$	$= \sin 5$
v_4	$= v_1 + v_2$	$= 0.693 + 10$
v_5	$= v_4 - v_3$	$= 10.693 + 0.959$
y	$= v_5$	$= 11.652$

Also, we have a computational graph to generate the computing order



Therefore, we must check how to build the graph.

3.2 Creating the Computational Graph

A graph consists of nodes and edges. We must discuss what a node/edge is and how to store information. From the graph shown above, we see that each node represents an intermediate expression:

$$\begin{aligned}
 v_1 &= \log x_1, \\
 v_2 &= x_1 \times x_2, \\
 v_3 &= \sin x_2, \\
 v_4 &= v_1 + v_2, \\
 v_5 &= v_4 - v_3.
 \end{aligned}$$

The expression in each node is from an operation to expressions in other nodes. Therefore, it is natural to construct an edge

$$u \rightarrow v,$$

if the expression of a node v is based on the expression of another node u . We say node u is a parent node (of v) and node v is a child node (of u). To do the forward calculation, at node v we should

store v 's parents. Additionally, we need to record the operator applied on the node's parents and the resulting value. For example, the construction of the node

$$v_2 = x_1 \times x_2,$$

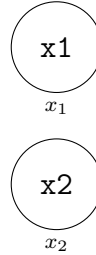
requires to store v_2 's parent nodes $\{x_1, x_2\}$, the corresponding operator “ $\times$ ” and the resulting value. Up to now, we can implement each node as a class **Node** with the following members.

member	data type	example for Node v_2
numerical value	float	10
parent nodes	List[Node]	$[x_1, x_2]$
child nodes	List[Node]	$[v_4]$
operator	string	"mul" (for $\times$)

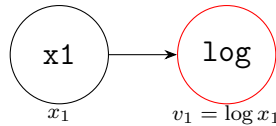
At this moment, it is unclear why we should store child nodes in our **Node** class. Later we will explain why such information is needed. Once the **Node** class is ready, starting from initial nodes (which represent x_i 's), we use nested function calls to build the whole graph. In our case, the graph for $y = f(x_1, x_2)$ can be constructed via

`y = sub(add(log(x1), mul(x1, x2)), sin(x2)).`

let us see this process step by step and check what each function must do. First, our starting point is the root nodes created by the **Node** class constructor.



These root **Nodes** have empty members “parent nodes,” “child nodes,” and “operator” with only “numerical value” respectively set to x_1 and x_2 . Then, we apply our implemented `log(node)` to the node **x1**.

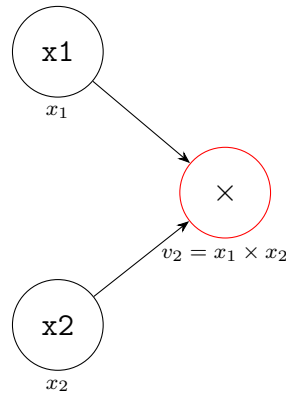


The implementation of our `log` function should create a **Node** instance to store $\log(x_1)$. Therefore, what we have is a wrapping function that does more than the log operation; see details in Section 3.3. The created node is the v_1 node in our computational graph. Next, we discuss details of the node creation. From the current `log` function and the input node x_1 , we know contents of the following members:

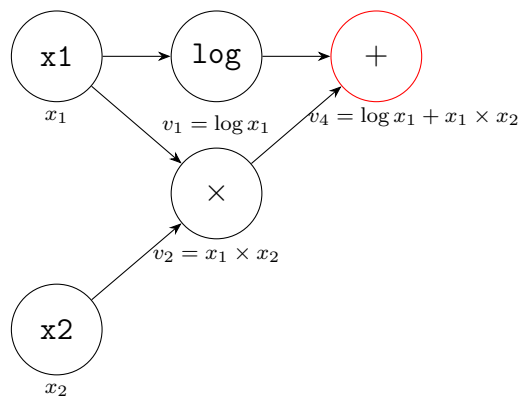
- parent nodes: $[x_1]$
- operator: "log"
- numerical value: $\log 2$

However, we have no information about children of this node. The reason is obvious because we have not had a graph including its child nodes yet. Instead, we leave this member “child nodes” empty and let child nodes to write back the information. By this idea, our `log` function should add v_1 to the “child nodes” of x_1 . See more details later in Section 3.3.

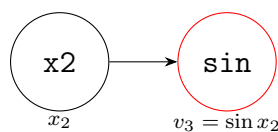
We move on to apply `mul(node1, node2)` on nodes `x1` and `x2`.



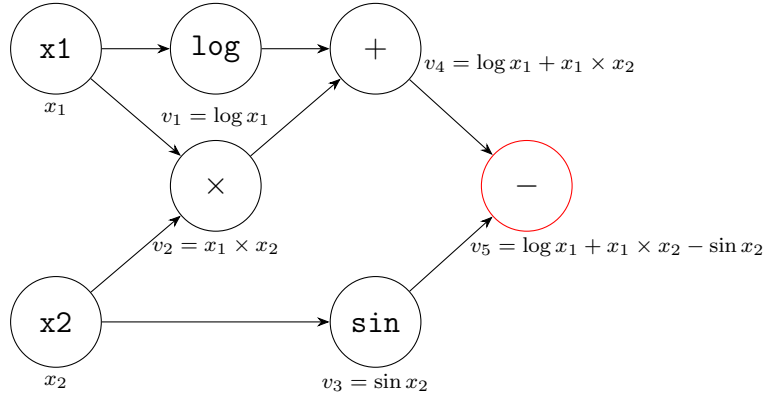
Similarly, the `mul` function generates a `Node` instance. However, different from `log(x1)`, the node created here stores two parents (instead of one). Then we apply the function call `add(log(x1), mul(x1, x2))`.



Next, we apply `sin(node)` to `x2`.



Last, applying `sub(node1,node2)` to the output nodes of `add(log(x1), mul(x1, x2))` and `sin(x1)` leads to



We can conclude that each function generates exactly one `Node` instance; however, the generated nodes differ in the operator, the number of parents, etc.

3.3 Wrapping Functions

We mentioned that a function like “mul” does more than calculating the product of two numbers. Here we show more details. These customized functions “add”, “mul” and “log” in the previous pages are *wrapping* functions, which “wrap” numerical operations with additional codes. An important task is to maintain the relation between the constructed node and its parents/children. This way, the information of graph can be preserved.

For example, we may implement the following “mul” function.

Listing 1: The wrapping function “mul”

```

def mul(node1, node2):
    value = node1.value * node2.value
    parent_nodes = [node1, node2]
    newNode = Node(value, parent_nodes, "mul")
    node1.child_nodes.append(newNode)
    node2.child_nodes.append(newNode)
    return newNode

```

In this code, we add the created node to the “child nodes” lists of the two input nodes: `node1` and `node2`. As we mentioned earlier, when `node1` and `node2` were created, their lists of child nodes were unknown and left empty. Thus, in creating each node, we append the node to the list of its parent(s).

The output of the function should be the created node. This setting enables the nested function call. That is, calling

$$y = \text{sub}(\text{add}(\text{log}(x1), \text{mul}(x1, x2)), \text{sin}(x2))$$

finishes the function evaluation. At the same time, we build the computational graph.

4 Topological Order and Partial Derivatives

Once the computational graph is built, we want to use the information in the graph to compute

$$\frac{\partial y}{\partial x_1} = \frac{\partial v_5}{\partial x_1}.$$

4.1 Finding the Topological Order

Recall that $\partial v / \partial x_1$ is denoted by $\dot{v}$. From the chain rule,

$$\dot{v}_5 = \frac{\partial v_5}{\partial v_4} \dot{v}_4 + \frac{\partial v_5}{\partial v_3} \dot{v}_3. \quad (6)$$

We are able to calculate

$$\frac{\partial v_5}{\partial v_4} \text{ and } \frac{\partial v_5}{\partial v_3}, \quad (7)$$

because the node v_5 stores the needed information related to its parents v_4 and v_3 . We defer the details on calculating (7), so the focus is now on calculating $\dot{v}_4$ and $\dot{v}_3$. For $\dot{v}_4$, we further have

$$\dot{v}_4 = \frac{\partial v_4}{\partial v_1} \dot{v}_1 + \frac{\partial v_4}{\partial v_2} \dot{v}_2, \quad (8)$$

which, by the same reason, indicates the need of $\dot{v}_1$ and $\dot{v}_2$. On the other hand, we have $\dot{v}_3 = 0$ since v_3 (i.e., $\sin(x_2)$) is not a function of x_1 . The discussion on calculating $\dot{v}_4$ and $\dot{v}_3$ leads us to find that

$$v \text{ is not reachable from } x_1 \text{ in the graph} \Rightarrow \dot{v} = 0. \quad (9)$$

We say a node v is reachable from a node u if there exists a path from u to v in the graph. From (9), now we only care about nodes reachable from x_1 . Further, we must properly order nodes reachable from x_1 so that, for example, in (4.2), $\dot{v}_4$ and $\dot{v}_3$ are ready before calculating $\dot{v}_5$. Similarly, $\dot{v}_1$ and $\dot{v}_2$ should be available when calculating $\dot{v}_4$.

To consider nodes reachable from x_1 , from the whole computational graph $G = \langle V, E \rangle$, where V and E are respectively sets of nodes and edges, we define

$$V_R = \{v \in V \mid v \text{ is reachable from } x_1\}$$

and

$$E_R = \{(u, v) \in E \mid u \in V_R, v \in V_R\}.$$

Then,

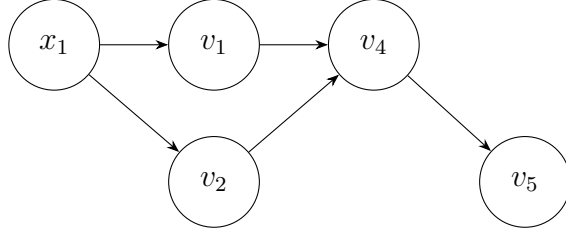
$$G_R \equiv \langle V_R, E_R \rangle$$

is a subgraph of G . For our example, G_R is the following subgraph with

$$V_R = \{x_1, v_1, v_2, v_4, v_5\}$$

and

$$E_R = \{(x_1, v_1), (x_1, v_2), (v_1, v_4), (v_2, v_4), (v_4, v_5)\}.$$



We aim to find a “suitable” ordering of V_R satisfying that each node $u \in V_R$ comes before all of its child nodes in the ordering. By doing so, u can be used in the derivative calculation of its child nodes; see (8). For our example, a “suitable” ordering can be

$$x_1, v_1, v_2, v_4, v_5.$$

In graph theory, such an ordering is called a *topological ordering* of G_R . Since G_R is a directed acyclic graph (DAG), a topological ordering must exist.⁶ We may use depth first search (DFS) to traverse G_R to find the topological ordering. For the implementation, earlier we included a member “child nodes” in the `Node` class, but did not explain why. The reason is that to traverse G_R from x_1 , we must access children of each node.

Based on the above idea, we can have the following code to find a topological ordering.

Listing 2: Using depth first search to find a topological ordering

```

def topological_order(rootNode):
    def add_children(node):
        if node not in visited:
            visited.add(node)
            for child in node.child_nodes:
                add_children(child)
            ordering.append(node)
    ordering, visited = [], set()
    add_children(rootNode)
    return list(reversed(ordering))

```

The function `add_children` implements the depth-first-search of a DAG. From the input node, it sequentially calls itself by using each child as the input. This way explores all nodes reachable from the input node. After that, we append the input node to the end of the output list. Also, we must maintain a set of visited nodes to ensure that each node is included in the ordering exactly once. For our example, the input in calling the above function is x_1 , which is also the root node of G_R . The left-most path of the depth-first search is

$$x_1 \rightarrow v_1 \rightarrow v_4 \rightarrow v_5, \tag{10}$$

so v_5 is added first. In the end, we get the following list

$$[v_5, v_4, v_1, v_2, x_1].$$

⁶We do not get into details, but a proof can be found in Kleinberg and Tardos (2005).

Then, by reversing the list, a node always comes before its children. One may wonder whether we can add a node to the list before adding its child nodes. This way, we have a simpler implementation without needing to reverse the list in the end. However, this setting may fail to generate a topological ordering. We obtain the following list for our example:

$$[x_1, v_1, v_4, v_5, v_2].$$

A violation occurs because v_2 does not appear before its child v_4 . The key reason is that in a DFS path, a node may point to another node that was added earlier through a different path. Then this node becomes after one of its children. For our example,

$$x_1 \rightarrow v_2 \rightarrow v_4 \rightarrow v_5$$

is a path processed after the path in (10). Thus, v_2 is added after v_4 and a violation occurs. Reversing the list can resolve the problem. To this end, in the function `add_children`, we must append the input node in the end.

In automatic differentiation, methods based on the topological ordering are called *tape-based* methods. They are used in some real-world implementations such as Tensorflow. The ordering is regarded as a tape. We read the nodes one by one from the beginning of the sequence (tape) to calculate the derivative value.

Based on the obtained ordering, subsequently let us see how to compute each $\dot{v}$.

4.2 Computing the Partial Derivative

Earlier, by the chain rule, we have

$$\dot{v}_5 = \frac{\partial v_5}{\partial v_4} \dot{v}_4 + \frac{\partial v_5}{\partial v_3} \dot{v}_3.$$

In Section 4.1, we mentioned that

$$\dot{v}_4 \text{ and } \dot{v}_3$$

should be ready before calculating $\dot{v}_5$. For

$$\frac{\partial v_5}{\partial v_4} \text{ and } \frac{\partial v_5}{\partial v_3},$$

we are able to calculate and store them when v_5 is created. The reason is that from

$$v_5(v_4, v_3) = v_4 - v_3,$$

we know

$$\frac{\partial v_5}{\partial v_4} = 1 \text{ and } \frac{\partial v_5}{\partial v_3} = -1.$$

A general form of our calculation is

$$\dot{v} = \sum_{u \in v's \text{ parents}} \frac{\partial v}{\partial u} \dot{u}. \quad (11)$$

The second term, $\dot{u} = \frac{\partial u}{\partial x_1}$, comes from past calculation due to the topological ordering. We can calculate the first term because u is one of v 's parent(s) and we know the operation at v . For example, we have $v_4 = v_1 \times v_2$, so

$$\frac{\partial v_4}{\partial v_1} = v_2 \text{ and } \frac{\partial v_4}{\partial v_2} = v_1.$$

These values can be immediately computed and stored when we construct the computational graph. Therefore, we add one member “gradient w.r.t. parents” to our `Node` class. In addition, we need a member “partial derivative” to store the accumulated sum in the calculation of (11). In the end, this member stores the $\dot{v}_i$ value. Details of the derivative evaluation are in Listing 4. The complete list of members of our node class is in the following table.

member	data type	example for Node v_2
numerical value	float	10
parent nodes	List[Node]	$[x_1, x_2]$
child nodes	List[Node]	$[v_4]$
operator	string	"mul"
gradient w.r.t parents	List[float]	$[5, 2]$
partial derivative	float	5

We update the `mul` function accordingly.

Listing 3: The wrapping function “mul”. The change from Listing 1 is in red color.

```
def mul(node1, node2):
    value = node1.value * node2.value
    parent_nodes = [node1, node2]
    newNode = Node(value, parent_nodes, "mul")
    newNode.grad_wrt_parents = [node2.value, node1.value]
    node1.child_nodes.append(newNode)
    node2.child_nodes.append(newNode)
    return newNode
```

As shown above, we must compute

$$\frac{\partial \text{newNode}}{\partial \text{parentNode}}$$

for each parent node in constructing a new child node. Here are some examples other than the `mul` function:

- For `add(node1, node2)`, we have

$$\frac{\partial \text{newNode}}{\partial \text{Node1}} = \frac{\partial \text{newNode}}{\partial \text{Node2}} = 1,$$

so the red line is replaced by

```
newNode.grad_wrt_parents = [1., 1.].
```

- For $\log(\text{node})$, we have

$$\frac{\partial \text{newNode}}{\partial \text{Node}} = \frac{1}{\text{Node.value}},$$

so the red line becomes

```
newNode.grad_wrt_parents = [1/node.value].
```

Now, we know how to get each term in (11), i.e., the chain rule for calculating $\dot{v}$. Therefore, if we follow the topological ordering, all $\dot{v}_i$ (i.e., partial derivatives with respect to x_1) can be calculated. An implementation to compute the partial derivatives is as follows. Here we store the resulting value in the member `partial_derivative` of each node.

Listing 4: Evaluating derivatives

```
def forward(rootNode):
    rootNode.partial_derivative = 1
    ordering = topological_order(rootNode)
    for node in ordering[1:]:
        partial_derivative = 0
        for i in range(len(node.parent_nodes)):
            dnode_dparent = node.grad_wrt_parents[i]
            dparent_droot = node.parent_nodes[i].partial_derivative
            partial_derivative += dnode_dparent * dparent_droot
        node.partial_derivative = partial_derivative
```

4.3 Summary

The procedure for forward mode includes three steps:

1. Create the computational graph
2. Find a topological order of the graph associated with x_1
3. Compute the partial derivative with respect to x_1 along the topological order

We discuss not only how to run each step but also what information we should store. This is a minimal implementation to demonstrate the forward mode automatic differentiation.

5 Implementation of Reverse Mode

In Section 2.2, we introduced the basic concept of reverse mode automatic differentiation. Now we show how to implement it. Recall that in reverse mode, we denote

$$\bar{v} = \frac{\partial y}{\partial v}$$

as the derivative of the output y with respect to the variable v . This is different from forward mode, where we compute $\dot{v} = \partial v / \partial x_1$. The key advantage of reverse mode is that we can compute $\partial y / \partial x_i$ for all input variables x_i in a single pass.

5.1 Reverse Topological Order

In forward mode, we needed a topological ordering that ensured each node came before its children. For reverse mode, we need the opposite: each node in the ordering should come before its parents. This makes sense because we start from the output $y = v_5$ and propagate gradients backward to the inputs.

Now, for reverse mode starting from the output node y , we define the set of nodes that can reach the output:

$$V_B = \{v \in V \mid y \text{ is reachable from } v\}$$

and correspondingly

$$E_B = \{(u, v) \in E \mid u \in V_B, v \in V_B\}.$$

Then $G_B \equiv \langle V_B, E_B \rangle$ is a subgraph of G containing all nodes that contribute to the output.

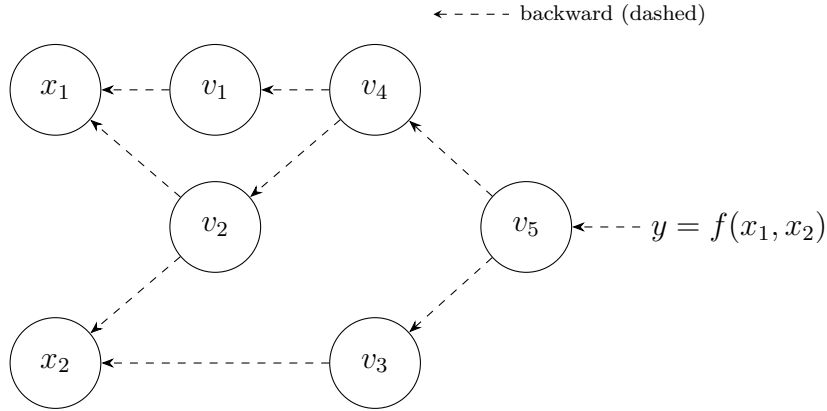
To perform reverse mode, we conceptually reverse all edges in G_B . We define the reversed edge set as

$$E_B^{\text{rev}} = \{(v, u) \mid (u, v) \in E_B\}.$$

The reversed graph is then

$$G_B^{\text{rev}} \equiv \langle V_B, E_B^{\text{rev}} \rangle.$$

In this reversed graph, edges point from children to parents. The dashed arrows show the direction of gradient flow in reverse mode:



Listing 5: Finding a reverse topological ordering in `simpleautodiff`

```
def reverse_topological_order(output_node):
    def add_parents(node):
        if node not in visited:
            visited.add(node)
            for parent in node.parent_nodes:
                add_parents(parent)
            ordering.append(node)
    ordering, visited = [], set()
    add_parents(output_node)
    return list(reversed(ordering))
```

To find a reverse topological ordering of G_B , we can apply a topological ordering algorithm to G_B^{rev} . The key insight is that we can reuse the existing topological ordering algorithm from Section 4.1. Since our `Node` class stores both parent and child information, we can traverse the graph in either direction. For reverse mode, we traverse via child edges, which correspond to the forward edges in the reversed graph G_B^{rev} .

5.2 Implementation of Reverse Mode

The implementation follows the procedure described in section 2.2. The important steps are:

1. Initialize all `partial_derivative` members to zero.
2. Seed the output node with gradient 1.
3. For each node in reverse topological order, propagate its gradient to its parents. The gradient contribution to each parent is the current node’s gradient (`v_bar`) multiplied by the local derivative (`local_grad`). We use “`+=`” to accumulate contributions, which is essential when a parent has multiple children.

Listing 6: The backward function for reverse mode in `simpleautodiff`

```
def backward(output_node):
    ordering = reverse_topological_order(output_node)
    for node in ordering:
        node.partial_derivative = 0.0
    output_node.partial_derivative = 1.0
    for node in ordering:
        v_bar = node.partial_derivative
        for i, parent in enumerate(node.parent_nodes):
            local_grad = node.grad_wrt_parents[i]
            contrib = v_bar * local_grad
            parent.partial_derivative += contrib
```

Notice that we reuse the same `Node` class and the same member `partial_derivative` that we used for forward mode. However, the meaning is different:

- In forward mode: `partial_derivative` stores $\dot{v} = \partial v / \partial x_1$.
- In reverse mode: `partial_derivative` stores $\bar{v} = \partial y / \partial v$.

This is possible because we run either forward mode or reverse mode, but not both at the same time.

6 Implementation of Autograd

While `simpleautodiff` illustrates the fundamental principles of automatic differentiation, production libraries like `autograd` introduce additional architectural layers to handle real-world complexity. We now analyze the internal mechanics of the `autograd` library, which is for example used by Pytorch.

6.1 The Internal Architecture

`Autograd` decouples the responsibilities of data storage and graph structure. To understand how the library functions, we must look at its three core classes: `Box`, `VJPNode`, and `Trace`.

6.1.1 The Box

The entry point to the library is the `Box` class. When a user differentiates a function, the input variables are wrapped in a `Box`. This class plays a role similar to the `Node` in `simpleautodiff` regarding data storage, but it is strictly a wrapper. It holds:

1. **The Value:** The actual underlying data (e.g., the NumPy array).
2. **The Trace:** A reference to the active recording context (explained below).
3. **The Node:** A reference to the `VJPNode` associated with this value.

Primitive operations (such as `numpy.add`) are wrapped to inspect their input arguments at runtime. If a `Box` instance is detected, it automatically triggers the mechanism to extend the computational graph.

6.1.2 The VJPNode

While the `Box` flows through the program like a normal variable, the `VJPNode` is left behind to form the computational graph. This is the direct equivalent of the graph-structure aspect of our `simpleautodiff.Node`. However, instead of storing scalar gradients, the `VJPNode` stores the implementation of the **Transposed Jacobian-vector Product** (introduced in Section 2.2.2). In the `autograd` codebase, this is simply referred to as the **VJP**.

During the forward pass, when a primitive operation occurs:

1. A new `VJPNode` is created.
2. It records its `parents` (the `VJPNodes` of the inputs).
3. The node identifies the registered formula for calculating $J^T r$ for the current operation (e.g., retrieving the rule that the VJP for $y = \sin(x)$ involves multiplication by $\cos(x)$).
4. It constructs a specific function to compute this product for the current instance. Crucially, this function “captures” (saves) the input values from the forward pass. For example, if we compute $\sin(2.0)$, the node saves the value 2.0 so it can compute the product using $\cos(2.0)$ later during the backward pass.

The resulting function is stored in the `vjp` attribute of the node. Conceptually, `node.vjp` is a **closure**, which is a function that retains access to the variables present when it was created.

6.1.3 The Trace

The `Trace` class is essential for managing the scope of differentiation. In simple implementations, we might assume there is only one global graph being built. However, advanced use cases often require computing derivatives of derivatives (higher-order differentiation). This leads to nested function calls where multiple differentiation passes are active simultaneously.

The `Trace` acts as a unique identifier for a specific differentiation pass.

1. **Scoping:** When a user calls the `grad` function, a new `Trace` object is created.
2. **Filtering:** Operations only affect the graph if the input `Box` belongs to the current top-level `Trace`.
3. **Safety:** This prevents operations from a higher-order derivative calculation from accidentally polluting the graph of a lower-order calculation.

In summary, while the `VJPNode` represents a single point in the graph, the `Trace` represents the active graph building session itself.

6.2 Finding the Order: Reference Counting

To perform the backward pass, we need to traverse the graph of `VJPNodes`. As established in Section 5.1, we require a reverse topological order. `Autograd` implements this in `autogradutil.py` using a `toposort` function based on reference counting, an algorithm introduced by Kahn (1962).

This iterative algorithm ensures that we strictly follow the dependencies defined in the computational graph:

1. **Counting Children:** The function first traverses the graph starting from the `end_node` (the output) backwards to the inputs via the `parents` attribute. It populates a dictionary called `child_counts` that records how many children each node has. This effectively counts how many times a node is used as an input for other operations.
2. **Queue Initialization:** It identifies nodes that have no remaining dependencies preventing them from being processed. Initially, only the `end_node` meets this criterion, as it has no children in the computational graph.
3. **Iterative Yielding:** The sorter removes a node from the queue and yields it for the backward pass. It then looks at the node's parents. For each parent, it decrements the parent's child count. If a parent's count reaches zero, it implies all its children have been processed, so the parent is added to the queue.

This iterative approach is robust against the deep recursion limits of Python, which is an improvement over the recursive DFS used in `simpleautodiff`.

Listing 7: Algorithmic logic of `autograd`'s topological sort

```
def toposort(end_node, parents=operator.attrgetter("parents")):
    child_counts = {}
    stack = [end_node]
    while stack:
        node = stack.pop()
        if node in child_counts:
            child_counts[node] += 1
        else:
            child_counts[node] = 1
            stack.extend(parents(node))

    childless_nodes = [end_node]
    while childless_nodes:
        node = childless_nodes.pop()
        yield node
        for parent in parents(node):
```

```

    if child_counts[parent] == 1:
        childless_nodes.append(parent)
    else:
        child_counts[parent] -= 1

```

6.3 The Backward Pass: Executing VJPs

With the topological order established, `autograd` executes the backward pass. This is defined in the `backward_pass` function in `core.py`. Instead of modifying nodes in-place, the function uses a local dictionary, `outgrads`, to store the accumulated gradients.

Listing 8: The backward pass implementation from `core.py`

```

def backward_pass(g, end_node):
    outgrads = {end_node: (g, False)}
    for node in toposort(end_node):
        outgrad = outgrads.pop(node)
        ingrads = node.vjp(outgrad[0])
        for parent, ingrad in zip(node.parents, ingrads):
            outgrads[parent] = add_outgrads(outgrads.get(parent), ingrad)
    return outgrad[0]

```

Let us analyze the specific steps in this production code:

- **1. Initialization with Metadata:** The dictionary is initialized with `{end_node: (g, False)}`. Unlike our simple implementation, `autograd` stores tuples. The boolean flag (here set to `False`) tracks whether the gradient object is mutable, allowing for memory optimizations during accumulation.
- **2. Memory Management:** Inside the loop, `outgrad = outgrads.pop(node)` retrieves the accumulated gradient and immediately removes it from the dictionary. This is simply for memory optimization.
- **3. The VJP Call:** The code calls `node.vjp(outgrad[0])`. We access index 0 to get the actual gradient object from the tuple. This executes the closure created during the forward pass, transforming the output gradient into input gradients via the Vector-Jacobian Product.
- **4. Robust Accumulation:** The loop iterates over parents. The accumulation is handled by `add_outgrads`. We pass `outgrads.get(parent)`, which returns `None` if the parent has not yet received a gradient. The `add_outgrads` helper function handles the logic of initializing the sum or adding to an existing value.
- **5. Return:** The topological sort yields the input node (the start of the graph) last. The variable `outgrad` retains this last popped value. Finally `outgrad[0]` is returned to provide the user with the final gradient array for the input variable.

This implementation cleanly separates the graph topology (handled by `toposort`) from the numerical calculus (handled by `vjp`) and the accumulator logic (handled by `add_outgrads`), resulting in a highly modular system.

7 Comparing `simpleautodiff` and `Autograd`

Having examined both the educational implementation (`simpleautodiff`) and the production library (`autograd`), we can now systematically compare their architectures. In this section, we identify three shortfalls of our educational implementation and how they are solved in `autograd`.

7.1 Core Differences

The gap between the educational implementation and the production library can be categorized into three architectural differences:

1. **Scalar vs. Vector Differentiation:** `simpleautodiff` operates exclusively on scalar values. Consequently, the local derivative is simply a single floating-point number. In contrast, `autograd` handles n -dimensional arrays (tensors). When variables u and v are vectors, the derivative theoretically represents a Jacobian matrix.
2. **Lack of Optimization:** `simpleautodiff` performs arithmetic operations directly within the Python interpreter using standard Python objects. Each addition or multiplication incurs the overhead of dynamic typing and object creation. Furthermore, it calculates gradients individually for every scalar, failing to exploit the parallelism available in modern CPUs.
3. **Handling higher derivatives:** `simpleautodiff` relies on mutating the state of a single global graph. The backward pass calculates gradients by updating the `partial_derivative` attribute of nodes in-place. Consequently, calculating higher-order derivatives (e.g., the Hessian) is impossible for two reasons. First, the backward pass performs pure arithmetic without creating new nodes, so no graph exists to represent the calculation of the derivative itself. Second, nested calls would attempt to write to the same `partial_derivative` field, causing data collisions.

7.2 Overcoming Implementation Challenges

`Autograd` can handle the three limitations listed above. Here we analyze the mechanisms used to resolve them.

7.2.1 Handling Vector Inputs via VJPs

To handle the vector inputs described in Section 7.1, **autograd** implements the mathematical strategy established in Section 2.2.2 directly into its architecture. Instead of calculating and storing derivative coefficients, **autograd** stores **executable functions**.

Specifically, the library replaces the explicit Jacobian matrix with the implicit Transposed Jacobian-vector Product (VJP). For every primitive operation, the library registers a specialized closure via `defvjp`. This closure encapsulates the linear algebra required to propagate the gradient. For instance, for a matrix multiplication $Y = XW$, the stored VJP function simply executes the matrix product $\bar{X} = \bar{Y}W^T$. By storing the *operation* rather than the *matrix*, **autograd** handles high-dimensional vectors with minimal memory overhead.

7.2.2 Accelerating Code Execution

Autograd achieves acceleration by delegating the heavy lifting to **Primitives**. The library wraps the highly optimized NumPy library. When a user performs an addition or dot product:

1. **The Overhead:** Python only handles the graph construction (creating the `VJPNode` and referencing parents). This is a constant-time overhead per operation.
2. **The Computation:** The actual numerical calculation is passed immediately to NumPy, which executes it in optimized C or Fortran.

Because the VJP functions (the backward pass logic) also rely on these NumPy primitives, the backward pass benefits from the same vectorization. This ensures that while the orchestration happens in Python, the numerical throughput is comparable to hand-written C code.

7.2.3 Handling Scopes via Traces

To address the issue of differentiation scope and enable higher-order derivatives, **autograd** leverages the **Trace** architecture introduced in Section 6.1.3. The fundamental limitation in **simpleautodiff** was its reliance on a singleton global graph, which leads to data collisions if a backward pass attempts to write to nodes that are currently being read or modified by another derivative process.

Autograd resolves this by using the **Trace** object to enforce strict isolation between different differentiation passes:

1. **Trace Stacking:** When `grad` is called recursively (e.g., for a Hessian), a new **Trace** is pushed onto a global stack.
2. **Graph Isolation:** As described in Section 6.1, input variables are wrapped in **Box** objects tagged with a specific Trace ID. Operations are only recorded on the graph if the input **Box** matches the currently active **Trace**.

3. **Resolution of Collisions:** This mechanism ensures that the backward pass of an *inner* derivative (which generates new numerical operations) is simply treated as a standard forward pass by the *outer* trace. The outer trace records these operations without interfering with the inner trace’s internal graph structure.

By replacing the fragile global state of our educational model with this stack-based scoping, `autograd` supports arbitrary nesting of derivatives (differentiation of differentiation) without any risk of variable collision.

7.3 Reverse Mode Summary

The procedure for reverse mode automatic differentiation includes three steps:

1. Create the computational graph during the forward pass. In this step, we record the dependencies (parents) and the local gradient information.
2. Find a reverse topological order of the graph starting from the output node.
3. Compute the partial derivatives with respect to the output along the reverse order.

We discussed how the implementation in Section 5 demonstrates the fundamental accumulation logic, while the production architecture in Sections 6 and 7 extends this to handle tensor operations, optimization, and complex dependency scopes.

Acknowledgements

This work was supported by National Science and Technology Council of Taiwan grant 110-2221-E-002-115-MY3.

References

- M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng. Tensorflow: A system for large-scale machine learning. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 265–283, 2016.
- M. Bartholomew-Biggs, S. Brown, B. Christianson, and L. Dixon. Automatic differentiation of algorithms. *Journal of Computational and Applied Mathematics*, 124(1-2):171–190, 2000.
- A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018.

- S. Chinchalkar. The application of automatic differentiation to problems in engineering analysis. *Computer Methods in Applied Mechanics and Engineering*, 118(1-2):197–207, 1994.
- A. B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11):558–562, 1962. doi: 10.1145/368996.369025.
- J. Kleinberg and E. Tardos. *Algorithm Design*. Addison-Wesley Longman Publishing Co., Inc., 2005. ISBN 0321295358.
- C. C. Margossian. A review of automatic differentiation and its efficient implementation. *Wiley interdisciplinary reviews: data mining and knowledge discovery*, 9(4):e1305, 2019.
- A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer. Automatic differentiation in Pytorch. 2017.